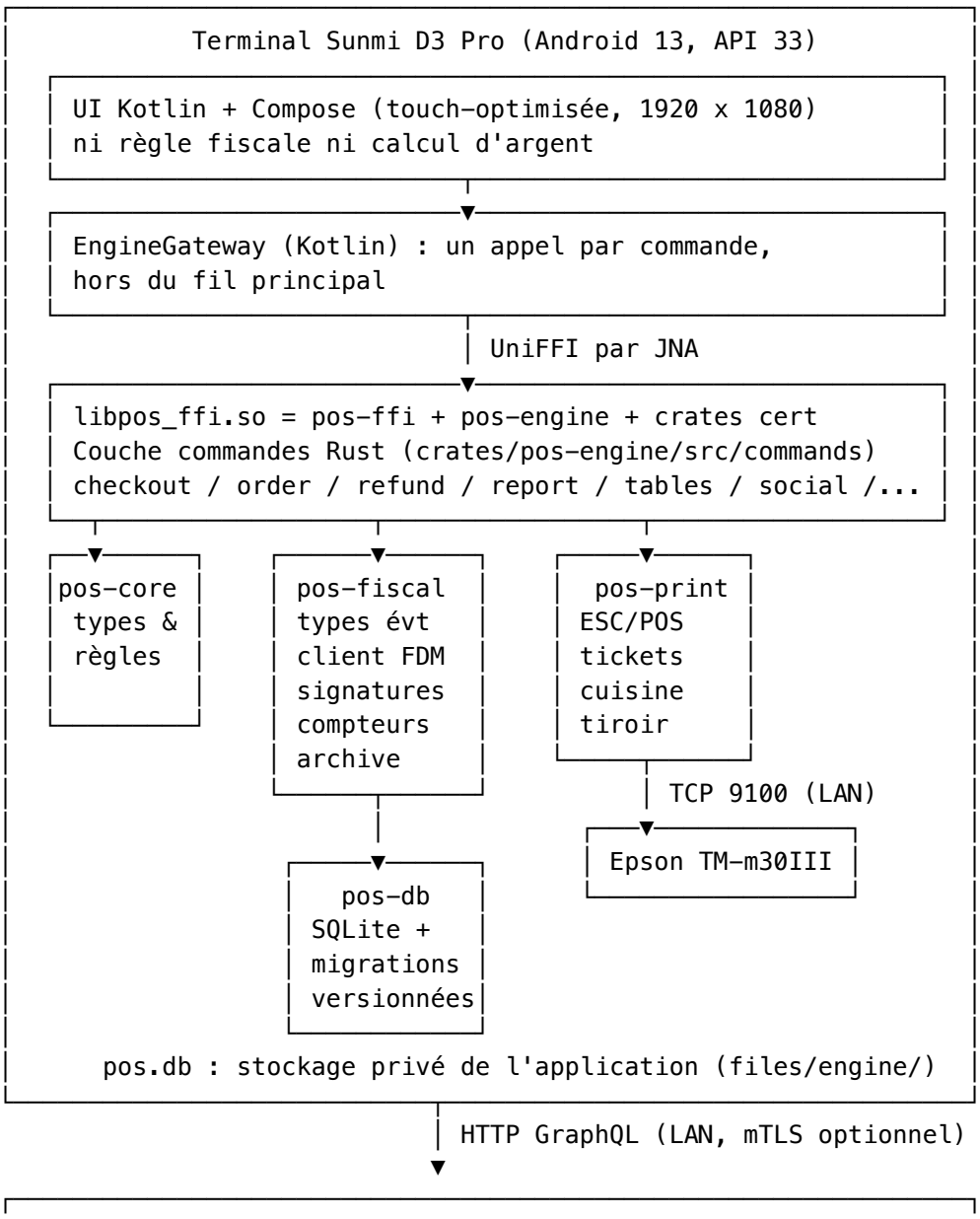
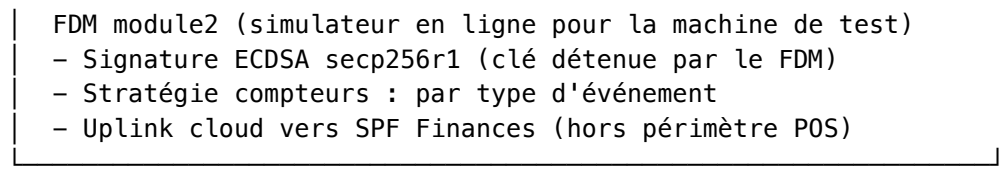
Document de processus technique - pili-pos-android SCE 2.0

Distributeur : Bruno Moyse / nuagemagique.dev (Bruxelles, BE) **Produit :** pili-pos-android 1.0.0 (posSwVersion pili-pos-android/1.0.0, build de soumission cert) **Plateforme :** Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google) **Clients cibles :** établissements HoReCa en Belgique (restaurants avec service en salle) **Date de soumission :** À effectuer (EV-18 : date de dépôt) **Specs implémentées :** POS v2.2 + FDM v1.4 + Addendum 1 + Basic Set v1.8 + Reports v1.3 + Advanced Set v1.6

1. Vue d'ensemble du système

`pili-pos-android` est une **application Android** : un seul APK, avec une interface en Kotlin et Compose et le moteur fiscal en Rust. Elle s'exécute sur le terminal Sunmi D3 Pro (Android 13, propriétaire de l'appareil, mode kiosque ; `PLATEFORME_ANDROID_FR.md` §2 et §3), se connecte par le réseau local à un FDM certifié SPF Finances, pilote une imprimante thermique ESC/POS Epson TM-m30III et persiste chaque événement fiscal dans une base SQLite locale append-only, dans le stockage privé de l'application. Le moteur est le même code source que celui de `pili-pos` sur Linux (`CADRE_DEMANDE_FR.md` §4).





Structure du code. Le noyau fiscal est le moteur `crates/pos-engine`, sans dépendance à Tauri ni à Android : les fonctions de commande (`crates/pos-engine/src/commands`), l'état partagé (`state.rs`), l'ouverture du moteur (`engine.rs` : base SQLite, migrations, reprise après arrêt brutal, chargement de la configuration, verrou exclusif sur le fichier `engine.lock` du répertoire de données) et les tâches de fond (`background.rs` : Z automatique, file d'envoi des e-mails, scrutation et file d'envoi HubRise, présence, verrouillage sur inactivité, suivi du FDM).

Sur Android, le moteur est enveloppé par deux couches. `android/rust/pos-ffi` est la façade UniFFI : une méthode par commande du moteur, qui appelle la même fonction `do_*` que l'enveloppe IPC Linux, sans logique fiscale propre (`docs/android/FFI.md`). Elle est compilée avec `pos-engine` (`default-features = false, features = ["cert"]`) en `libpos_ffi.so`, la seule bibliothèque native de l'APK (arm64-v8a). L'interface Kotlin et Compose appelle cette bibliothèque par UniFFI et JNA, uniquement au travers de `EngineGateway`, hors du fil principal. `EngineHolder` possède l'unique moteur du processus et un service au premier plan le garde ouvert tant que la caisse l'est (`docs/android/APP.md` §2). Le `posSwVersion` des événements et des tickets est une valeur que l'application transmet à l'ouverture du moteur ; sur Android elle est `pili-pos-android/` suivi de la version de l'APK. La configuration vient du lot de configuration signé du terminal (`MANUEL_INSTALLATION_FR.md`). Le harnais de tests `crates/pos-engine-testkit` ne fait pas partie de l'APK.

Règles qui gardent la logique fiscale dans le moteur. Aucune règle fiscale ni calcul d'argent n'existe en Kotlin : l'interface affiche, transmet et réagit aux événements, le moteur décide (`docs/android/ENGINE.md` §7).

- L'événement P à la sortie d'une commande : `commands::session::do_leave_order` (sans effet pour une vente à emporter ou une commande non ouverte), `do_set_active_order` (la commande à l'écran) et `do_lock_session` (verrouillage avec enregistrement de la commande active).
- Le pointage : l'entrée est mise en file par la connexion (`do_login`) et signée par la tâche de présence ; la sortie est signée par `commands::social::do_end_shift`, qui ferme toujours la session.
- Le verrouillage après 5 minutes d'inactivité : `commands::session::do_user_activity` et `background::run_idle_lock` ; l'interface se borne à transmettre l'activité de l'utilisateur, au plus une fois par seconde.
- Le suivi du FDM (intervalle, recul, classement de l'état) : `commands::fdm_monitor::do_fdm_health` et l'événement `fdm-status` ; l'interface reçoit le résultat.
- Le rendu en espèces, la ventilation de la TVA et l'aperçu du règlement : fonctions d'aide du moteur (`settlePreview`, `cashDueCents`, `vatSplit`). Un contrôle automatique (`scripts/android/p6_gate.sh`) interdit à la couche plateforme d'y toucher.
- La parité avec l'interface Linux : 75 des 84 commandes de l'interface Linux sont appelées par l'interface Compose et 9 sont exclues, avec leurs raisons (`LISTE_FONCTIONNALITES_FR.md` §7).

2. Comportement par événement

Événement	Déclencheur	Chemin de code POS	Notes
N (Vente normale)	L'opérateur tape "Payer" (sur place `dine_in` ou à emporter `takeaway`)	`PosEngine.checkout` (façade) → `commands/checkout::do_checkout` / `do_checkout_split` → `sell` → `signing::sign` : archive + clôture dans une seule tx SQLite	Blocage-sur-déconnexion-FDM imposé
P (Pro forma)	L'opérateur envoie la commande en cuisine, ou quitte la commande (règle du moteur, §1)	`commands/tables::do_sig` (appelée par `commands/session::do_leave`)	Table ouverte, pas d'incrément du compteur `N_order`
T (Formation)	L'opérateur est en mode formation	Comme N mais `is_training=true` dans l'input ; watermark UI	
C (Copie)	Réimpression d'un ticket antérieur	`commands/copy::do_copy`	Pointeur `original_ref` vers l'original
I (Facture)	Hors build Android 1.0	Code sous `#[cfg(feature = "peppol")]` (`commands/invoice.rs`)	Absent de la build Android (§8) ; événement optionnel dans SCE 2.0
F (Financier)	Mouvement caisse, ouverture seule du tiroir	`commands/financial::do_` / `do_open_drawer_fiscal`	Motif obligatoire pour ouverture tiroir
S (Social)	Connexion de l'opérateur (entrée), fin de service (sortie), règles du moteur (§1)	`commands/social::do_work_in` / `do_work_out` / `do_end_shift`	`INSZ` + n° employeur
R (Rapport)	Rapports Z et X	`commands/report::do_gen` / `do_generate_z_report`	Zéro effet sur `z_report_count` ; User-Z atomique

Chaque branche :

1. Charge les lignes pertinentes depuis SQLite (les requêtes sont dans `pos-db/src/queries/`).
2. Construit la struct d'input SCE-2.0 (`pos-fiscal::inputs::*`).
3. Sous le verrou de signature unique (`commands/signing.rs`), lit le prochain numéro de ticket, choisit la période d'ouverture, calcule la clé de doublon (voir §3.4) et le corps JSON canonique (voir §3.3), puis stocke la requête exacte dans `pending_signs`.
4. Appelle la méthode async `FdmClient::sign_*` correspondante (une requête restée sans réponse est d'abord renvoyée à l'identique, voir §4).
5. Sur succès : avance le compteur, ajoute à `fiscal_events`, ferme ou marque la commande et clôt la ligne `pending_signs`, dans une seule transaction SQLite.
6. Retourne les données de ticket à l'UI ; l'UI déclenche l'impression via `pos-print`.

3. Signature / vérification

3.1 Transport

- **Protocole :** GraphQL sur HTTP/HTTPS.
- **Endpoint :** configuré via `pos.json` (`fdm_url`), fourni par le lot de configuration signé.
- **Sondage de statut :** le moteur interroge `FdmClient::status` (`crates/pos-engine/src/commands/fdm_monitor.r` boucle de `background.rs`) dès qu'une session s'ouvre, puis toutes les 30 s tant que le FDM répond,

avec un recul jusqu'à 5 min tant qu'il ne répond pas ; l'interface ne fait que recevoir le résultat (événement `fdm-status`). Le checkout n'appelle pas le statut : chaque vente part directement vers `signing::sign` (§5).

3.2 Signature de réponse

- Algorithme : **ECDSA secp256r1**, calculé par le FDM avec sa propre clé. Le POS ne détient pas la clé publique du FDM et ne vérifie pas la signature ECDSA ; il la conserve telle quelle, avec la réponse complète, dans `fiscal_events.fdm_response_json`.
- Contrôle de cohérence avant archivage : `pos_fiscal::response_check::check_sign_result`, appelé par `FdmClient` pour chaque réponse de signature (y compris les deux moitiés du Z M184). Il exige l'écho de `posId`, `posFiscalTicketNo`, `posDateTime`, `deviceId` et `terminalId`, l'`eventOperation` et l'`eventLabel` de la mutation (T en formation), une `digitalSignature` en base64 valide et, pour N, une `shortSignature` égale à Hex(SHA1(Base64Decode(digitalSignature))) (`pos_fiscal::signature::compute_short_signature`) ainsi qu'une `verificationUrl`. Toute divergence produit l'erreur `FDM_RESPONSE_MISMATCH` : rien n'est archivé ni imprimé.
- La short signature apparaît sur le ticket imprimé (40 caractères hexadécimaux).

3.3 JSON canonique

- `pos-fiscal::canonical::to_canonical()` sérialise une valeur en JSON canonique style RFC 8785 : clés triées par codepoint Unicode à chaque niveau, pas d'espace insignifiant, nombres normalisés (pas de zéros traînants, `-0` → `0`).
- Utilisé pour **chaque** corps de requête envoyé au FDM, de sorte qu'un flux de signature reproductible par hash est possible.
- Test de propriété dans `crates/pos-fiscal/src/canonical.rs::tests` (vérification de déterminisme sur 50 itérations).

3.4 Clé de doublon FDM

- `pos-fiscal::inputs::IdempotencyKey { pos_id, pos_date_time, terminal_id, event_label, pos_fiscal_ticket_no }`.
- Chaque type d'input implémente `.idempotency_key()`.
- Règle de doublon du FDM : même clé et contenu identique, le FDM renvoie sa réponse déjà signée avec l'avertissement `DUPLICATE_REQUEST` ; même clé et contenu différent, il répond `INVALID_REQUEST`. Le POS ne reconstruit donc jamais une requête pour un numéro déjà envoyé : il renvoie la requête stockée, octet pour octet (§4).
- Les tests unitaires dans `crates/pos-fiscal/src/inputs.rs` couvrent l'égalité de la clé et le mapping de label par type.

4. Stratégie de compteurs

Voir `STRATEGIE_COMPTEURS_FR.md`. **Compteurs par type d'événement** stockés dans la table `fiscal_counters` ; le numéro est lu sous le verrou de signature et le compteur n'avance que dans la transaction SQLite qui archive l'événement signé, de sorte que :

- Les compteurs sont strictement monotones par type d'événement et deux opérations ne partagent jamais un numéro (verrou unique, `signing.rs`).
- Une requête non envoyée ou refusée par le FDM ne consomme pas son numéro - pas de trou.
- Une requête dont la réponse est perdue reste ouverte dans `pending_signs` ; elle est renvoyée à l'identique (même numéro, même contenu) avant toute autre signature, et le FDM renvoie sa réponse avec `DUPLICATE_REQUEST`. L'événement est archivé une fois, sous son premier numéro.
- Tant qu'elle n'est pas résolue, les autres signatures sont refusées (`FDM_ANSWER_PENDING`) ; la tâche de fond (60 s) renvoie la requête.

- Risques résiduels : un FDM qui aurait perdu sa mémoire des doublons signerait le renvoi comme un nouvel événement (détecté par les compteurs et journalisé, non évitable par le POS) ; une action de l'écran peut attendre jusqu'à 30 s derrière un Z automatique ou un renvoi (`signing_busy` ensuite).

5. Comportement hors-ligne / déconnexion

- **Pas de FDM, pas d'événement.** Chaque commande fiscale (vente, remboursement, commande, addition, copie, pointage, mouvement de caisse, ouverture de tiroir, rapport) fait signer son événement par le FDM *avant* d'écrire quoi que ce soit : il n'y a pas de pré-contrôle de statut, la vente passe directement par `signing::sign`. Si le FDM n'est pas joignable, `FdmError::outcome()` donne `NotSent` (la ligne `pending_signs` passe à `not_sent`) et la commande renvoie une erreur (`success: false`, « FDM not connected - cannot complete transaction » pour une vente, transmise à l'interface), rien n'est archivé, aucun compteur n'avance, la commande reste ouverte et le tiroir reste fermé. L'opérateur réessaie une fois le FDM revenu. Si la requête avait été envoyée et que la réponse est perdue, la requête est conservée et renvoyée à l'identique avant toute autre signature (§4) : l'événement n'est ni perdu ni doublé.
- **Aucun mode hors-ligne**, dans aucune build : le code ne contient aucun chemin qui archive ou imprime une vente non signée.
- L'interface affiche en permanence l'état du FDM (voyants `StatusLights.kt`, alertes `FdmBanner.kt`), interrogé par le moteur (`commands/fdm_monitor.rs`) toutes les 30 s avec recul jusqu'à 5 min, avec un message lisible pour chaque code d'erreur du FDM. Sans réponse du FDM, l'état passe à `connected = false`, gravité « bloquante ».
- Seule exception : les entrées de commande (P) non encore envoyées restent en attente dans `order_entries` et partent à l'enregistrement suivant (commande, addition ou vente) ; ce ne sont pas des événements tant que le FDM ne les a pas signées.

6. Pipeline d'impression

- Crate : `crates/pos-print`.
- Transport : `pos-print::transport` n'implémente que TCP brut (port 9100) vers la TM-m30III à une IP LAN statique. Pas de transport USB ni série.
- Layout : `pos-print::receipt` formate le ticket fiscal SCE-2.0 selon la spec POS v2.2 §III.3 - en-tête (nom restaurant, adresse, TVA), corps (lignes avec label TVA), table de répartition TVA, bloc de données de contrôle issu de la réponse FDM, QR Code (Model 2, ECC M, URL 38 caractères ; version 3, 29×29 modules - voir `TICKET_NUMERIQUE_FR.md` §11).
- Tickets de commande P (bar / cuisine) : layout séparé (`crates/pos-print/src/bar_ticket.rs::format_bar_tick` envoyé par `commands/bar_slip.rs`). Le ticket cuisine `kitchen_ticket::format_kitchen_ticket` ne sert qu'aux commandes en ligne.
- Texte externe (noms de produits, notes, adresses) : passé par `escpos::printable`, qui supprime les octets de contrôle (un ESC ou GS serait une commande d'imprimante) et translittère les caractères hors CP858.
- Ouverture tiroir : séquence ESC/POS `0x1B 0x70 0x00 0x19 0xFA` émise dès qu'un paiement `cash` clôture une vente.
- Sur Android, le moteur attend jusqu'à environ 2 s chaque réponse DLE EOT après une impression (500 ms sur Linux) : la réponse arrive souvent après 500 ms sur le réseau du terminal (mesure des essais préparatoires).

6bis. Pipeline d'envoi du ticket numérique (e-mail)

- Crate : `crates/pos-mail`, plus les tables `ticket_deliveries` et `original_prints` de `crates/pos-db` (migrations `005`/`006`).
- Rendu (`pos-mail::render`) : corps texte = `pos_print::render` appliqué au **même** `format_receipt` que le papier ; corps HTML + QR PNG en pièce jointe uniquement quand le FDM a renvoyé une URL de vérification (événements N).
- Transport (`pos-mail::tem`) : client HTTP vers l'API Scaleway Transactional Email (`X-Auth-Token`, région `fr-par` par défaut).

- File d'attente (`pos-mail::outbox`) : tentative immédiate bornée à 8 s à l'encaissement, puis réessais en tâche de fond avec recul exponentiel (`commands::delivery::run_outbox_once`, toutes les 30 s). Détail complet : `TICKET_NUMERIQUE_FR.md`.

6ter. Commandes en ligne

- Crates : `crates/pos-hubrise` (réception, file d'envoi, menu reflet), `crates/pos-catalog` (validation et application du catalogue), commandes du moteur `crates/pos-engine/src/commands/webshop.rs` (façade Android : méthodes `webshop_*` et `hubrise_*` de `pos-ffi`).
- Interrupteur par installation : `hubrise_enabled` dans `pos.json`, puis un Gérant dans Réglages ; journal en ajout seul `integration_switch_log` (migration 25). Coupé : aucune boucle ne contacte HubRise et aucune commande n'est acceptée ; une commande déjà signée reste soldable.
- Chaque commande en ligne se termine par un N signé sur ce terminal, au nom de l'employé qui l'accepte (P puis N si elle est payée à la remise). Détail : `COMMANDES_EN_LIGNE_FR.md`.

6quater. Export nocturne (facultatif)

- Crate : `crates/pos-export` (sauvegarde par l'API Online Backup sur une connexion en lecture seule, chiffrement age, copie statistique, manifeste, client HTTPS, file d'attente) ; intégration au moteur dans `crates/pos-engine/src/commands/export.rs` et une boucle de tâche de fond dans `background.rs`.
- Section `export` de `pos.json` ; absente, rien n'est créé et aucune connexion n'est ouverte.
- Déclenché après chaque rapport Z réel (jamais en formation) ; hors du verrou de signature, hors du verrou de la base des commandes : une vente ou un Z n'attend jamais l'export. Le journal légal reste local. Détail : `EXPORT_NOCTURNE_FR.md`.
- Sur Android, la boucle de fond tourne dans le moteur de l'APK ; les copies locales sont dans `files/engine/exports` (mémoire interne, jamais sur la carte SD) et l'état comme le nouvel essai sont réservés au Gérant (`PLATEFORME_ANDROID_FR.md` §5.1).

7. Persistance

- Moteur : **SQLite** (DB en un seul fichier, `pos.db` dans `files/engine/` du stockage privé de l'application, jamais sur la carte SD ; `PLATEFORME_ANDROID_FR.md` §5).
- Mode : WAL (`PRAGMA journal_mode=WAL`).
- Migrations : versionnées via `pos-db::migrate::run_migrations`. La table `_migrations_applied` suit `(version, name, applied_at)`. Le chemin de rétro-compatibilité pour la migration one-shot d'origine est dans `migrate::backfill_legacy_db`.
- Log fiscal append-only : table `fiscal_events`. Le schéma impose la contrainte CHECK sur `event_type` et des triggers SQLite refusent tout `DELETE` et toute réécriture (`fiscal_events_no_delete`, `fiscal_events_no_rewrite`) ; le code applicatif n'en émet d'ailleurs jamais. Voir `SECURITE_BD_FR.md`.
- Sauvegarde : après chaque rapport Z, juste après son archivage (`signing::follow_up` pour `ReportZ`, qui appelle `commands::report::backup_database_impl`), snapshot via l'API SQLite Online Backup. Un échec de sauvegarde est journalisé et n'annule pas le Z. Le moteur garde les 7 copies les plus récentes dans `files/engine/backups/` (`SECURITE_BD_FR.md` §5).

8. Configurations de build

Build	Features Cargo	Produit
Android (APK déclaré)	pos-ffi construit pos-engine avec default-features = false, features = ["cert"] (hubrise seul), profil android-release, cible aarch64-linux-android	libpos_ffi.so dans l'APK : commandes en ligne présentes, activées par installation (COMMANDES_EN_LIGNE_FR.md) ; sans facturation Peppol (DECISION_PEPPOL_AP_FR.md)
Développement	même bibliothèque, version suffixée –dev (adb actif)	jamais une version déclarée

Vérifications : le contrôle ffi_deps de scripts/android/p2_gate.sh (arbre de dépendances de pos-ffi : pos-engine présent, pos-peppol et tauri absents) et la reproductibilité de l'APK par scripts/android/check_repro.sh. Empreintes de l'APK, de libpos_ffi.so et construction reproduite : À effectuer (EV-01, EV-02, EV-05).

9. Carte des sources

Préoccupation	Crate / fichier
Types d'événements fiscaux & structs d'input	crates/pos-fiscal/src/inputs.rs
Client GraphQL FDM	crates/pos-fiscal/src/fdm_client.rs
JSON canonique	crates/pos-fiscal/src/canonical.rs
Short signature (SHA-1)	crates/pos-fiscal/src/signature.rs
Contrôle des réponses FDM	crates/pos-fiscal/src/response_check.rs
Compteurs fiscaux	crates/pos-fiscal/src/counter.rs
Archivage tickets	crates/pos-fiscal/src/archive.rs
Schéma base de données	crates/pos-db/src/schema.rs
Runner de migrations	crates/pos-db/src/migrate.rs
CRUD commandes	crates/pos-db/src/queries/orders.rs
Helpers requêtes rapport Z	crates/pos-db/src/queries/reports.rs
Moteur fiscal (ouverture, verrou, tâches de fond)	crates/pos-engine/src/{engine,background,state}.rs
Façade UniFFI du moteur (une méthode par commande)	android/rust/pos-ffi/src/
Passerelle Kotlin vers le moteur (état, événements, appels)	android/app/src/main/kotlin/dev/nuagemagique/pilipos/
Écrans Compose (caisse, encaissement, rapports, réglages, commandes en ligne)	android/app/src/main/kotlin/dev/nuagemagique/pilipos/
Propriétaire de l'appareil, export SPF, mise à jour, journal de plateforme	android/app/src/main/kotlin/dev/nuagemagique/pilipos/
Harnais de tests (hors binaire certifié)	crates/pos-engine-testkit/
Flux checkout	crates/pos-engine/src/commands/checkout.rs
Tables / split / merge	crates/pos-engine/src/commands/tables.rs
Remboursement + note de crédit	crates/pos-engine/src/commands/refund.rs
Rapports (X/Z)	crates/pos-engine/src/commands/report.rs
Social (événements S)	crates/pos-engine/src/commands/social.rs
Financier (événements F)	crates/pos-engine/src/commands/financial.rs
Copie (événements C)	crates/pos-engine/src/commands/copy.rs
Sérialiseur UBL Peppol	crates/pos-peppol/src/ubl.rs
AccessPoint Peppol	crates/pos-peppol/src/access_point.rs
Commandes Peppol (enveloppe IPC : src-tauri/src/ipc/invoice.rs)	crates/pos-engine/src/commands/invoice.rs

Préoccupation	Crate / fichier
Résolution du support + commandes ticket numérique	`crates/pos-engine/src/commands/delivery.rs`
Export nocturne : moteur (état, passage, commandes)	`crates/pos-engine/src/commands/export.rs`
Export nocturne : sauvegarde chiffrée, copie statistique, envoi, file d'attente	`crates/pos-export/src/`
Rendu e-mail (texte/HTML/QR)	`crates/pos-mail/src/render.rs`
Client Scaleway TEM	`crates/pos-mail/src/tem.rs`
File d'attente + réessais	`crates/pos-mail/src/outbox.rs`
Validation/masquage d'adresse	`crates/pos-mail/src/address.rs`
Requêtes `ticket_deliveries` / `original_prints`	`crates/pos-db/src/queries/{deliveries,prints}.rs`
Impression ESC/POS	`crates/pos-print/src/escpos.rs`
Layout ticket	`crates/pos-print/src/receipt.rs`
Transport imprimante	`crates/pos-print/src/transport.rs`

Les trois lignes Peppol ci-dessus désignent du code présent dans le dépôt mais absent de la build Android (§8).

10. Couche plateforme Android

Ces fonctions n'existent que sur Android et n'interviennent pas dans le calcul fiscal ; le moteur n'en dépend pas.

- **Propriétaire de l'appareil et kiosque** : restrictions système, Lock Task, politique de mise à jour du système, débogage coupé en version de production (`PLATEFORME_ANDROID_FR.md` §3).
- **Export pour le SPF** : copie de `pos.db`, des 7 sauvegardes et des journaux sur une clé USB ou une carte SD, à la demande du Gérant (`SECURITE_BD_FR.md` §8).
- **Mise à jour** : lancée par le Gérant, artefact déclaré seul (`GARANTIE_MAJ_LANGUES_FR.md` §2.1).
- **Chien de garde et reprise** : redémarrage de l'application après un état fatal du moteur, au plus 3 fois en 10 minutes (`PLATEFORME_ANDROID_FR.md` §8).
- **Journal d'audit de la plateforme** : fichier JSON Lines privé, distinct du journal fiscal de `pos.db` (`PLATEFORME_ANDROID_FR.md` §3.8).
- **Pas d'assistance à distance** : le moteur est ouvert sans programme de support, la commande de session d'assistance répond « non disponible » et l'interface n'en propose pas l'entrée (`PLATEFORME_ANDROID_FR.md` §7).